universidade de aveiro deti departamento de electrónica, telecomunicações e informática

BookMarket

Practical Work 1

Group 3
Beatriz Francisco - 118638
Joana Santiago - 119705
Matilde Rodrigues - 119714

Contents

1. Introduction **3**
2. Main Application Features **3**
2.1. Buyer 3
2.2. Seller 3
2.3. Admin 3
3. Django Features Used **4**
3.1. Core Architecture & Routing 4
3.2. User Authentication & Authorization 4
3.3. Data Management (ORM & Persistence) 4
3.4. Frontend Integration & UI 5
3.5. Administration & Monitoring 5
4. Access Information **5**
4.1. Deployment Link 5
4.2. Test Credentials 5
4.3. Local Setup 5
5. Conclusion **6**

1. Introduction

The second-hand book market represents a sustainable and economically accessible alternative to purchasing new literature. However, there's a lack of dedicated and trustworthy digital platforms for buying or selling this type of item, forcing the users to rely on generic marketplace solutions that don't offer the best features.

For that reason, **BookMarket** emerges as a response for this "problem". This solution is a web application developed in Django that implements a marketplace specialized in buying and selling second-hand books, with two different types of users, buyers and sellers. Additionally, there's a third user, the administrator (admin) that manages the accounts and the users.

Our platform was designed with the focus on an user experience, offering a variety of features such as an integrated chat system, star-rated reviews, analytical dashboard for sellers and an intuitive and simple purchase flow with simulated payment confirmation.

The report is organized as follows: description of the main features of the application in section 2, succeeded by the required information for access and platform configuration in section 3, and finishing with the conclusion and final reflections on the work made in section 4.

2. Main Application Features

2.1. Buyer

The Buyer is able to browse available books and view detailed listings, search for books by title and category, mark the favourites one, make purchases and track past purchases in a personal purchase history. They can also leave ratings and reviews for purchased books, use the in-app chat to contact sellers, manage buyer profile information and shipping addresses. Finally, there's the possibility to switch to the seller mode.

2.2. Seller

The Seller can publish new books with condition, images and price, mark books as disabled, and access a seller dashboard with sales analytics. They can also view recent sales, average ratings, active listing counts, see buyer reviews and feedback for sold books, communicate with buyers through the chat system, and manage their seller profile and payment details.

2.3. Admin

The Admin has access to a custom dashboard that shows platform statistics, such as sales over time and the registered users. Along with this data, the Admin can view individual user profiles to check their specific activity and metrics. To make management easier, each

user's page includes a direct link to the default Django Admin interface. This allows the Admin to quickly access the backend and make any necessary changes to the data.

3. Django Features Used

The following section outlines the technical stack and Django-specific features used to build the platform's infrastructure, ranging from database management to user authentication.

3.1. Core Architecture & Routing

- **Custom Application Modules:** The system logic is decoupled into specialized apps: **accounts** (user management), **books** (content management), **chat** (communication), and **stats** (data analysis).
- **URL Routing & Navigation:** Used Django's URL dispatcher with *include()* for modular routing, including custom redirects and development-specific media serving via *django.conf.urls.static*.
- **Middleware Stack:** Implemented standard and custom middleware to handle security, session persistence, CSRF protection and clickjacking mitigation.

3.2. User Authentication & Authorization

- **Custom User Model:** Extended the default authentication system by implementing a custom model (*AUTH_USER_MODEL = 'accounts.User'*) to meet specific project requirements.
- **Access Control:** Used the *login_required* decorator to restrict view access, ensuring that sensitive data and features are only available to authenticated users.
- **Built-in Auth Workflows:** Integrated *django.contrib.auth.urls* to handle standard procedures such as login, logout, and password management efficiently.

3.3. Data Management (ORM & Persistence)

- **Django ORM:** Managed database interactions through the Object-Relational Mapper, utilizing advanced querysets, filtering, ordering, and data aggregation.
- **Schema Versioning:** Used Django Migrations to track and apply database schema changes, ensuring consistency across different environments.
- **Session & Message Frameworks:** Employed *django.contrib.sessions* for state persistence and *django.contrib.messages* to provide real-time feedback to users during interactions.

3.4. Frontend Integration & UI

- **Template System:** Developed a dynamic UI using the Django Template Language (DTL), enhanced by custom context processors (*account_mode*) to inject global variables into templates.
- **Form Handling:** Implemented robust data validation and processing through Django Forms, specifically for registration (*RegisterForm*), profile updates (*UserEditForm*), and account management.
- **Static & Media Assets:** Configured *django.contrib.staticfiles* for managing CSS, JavaScript, and images, ensuring a streamlined asset pipeline.

3.5. Administration & Monitoring

- **Django Admin Interface:** Configured *django.contrib.admin* to provide a secure, web-based interface for administrators to manage application data and user records.

4. Access Information

4.1. Deployment Link

The BookMarket application is hosted on PythonAnywhere and deployed in the URL: https://bookmarket.pythonanywhere.com/ .

4.2. Test Credentials

The following accounts have been created to allow a full review of the platform and the difference between buyer, seller and admin views.

Type of user	Username	Password
Buyer	ferreirag07	g07buyer
Seller	patriciag	pg123seller
Buyer & Seller	francisca18	buyer&seller
Admin	admin	admin1

4.3. Local Setup

The project code can be obtained by cloning the repository from GitHub (https://github.com/BeatrizF26/TPW-Project-1). Following this step, a virtual environment

should be created before installing the necessary dependencies registered on the *requirements.txt* file.

The database and static files are then initialized using the following commands:

- *python3 manage.py migrate*
- *python3 manage.py collectstatic –noinput*

With these configurations complete, the application is ready to run locally. To start the development server, it is necessary to use *python3 manage.py runserver*, making the platform accessible at the local host address.

5. Conclusion

The development of this work proved to be an enriching practical exercise, allowing the consolidation of the knowledge in web development with Django and the versatility of this framework in building complete and fully functional web applications.

Through the development process we had attention to the user experience, especially to his design. Our goal was to build not only a functional platform, but also useful and intuitive for its users, and decisions such as the buyer/seller role switch within a single account, the seller analytics dashboard, and more, reflect it.

For the future, we could implement some features that would enrich this project: implement a book suggestion algorithm based on the buyer's purchase history and favourite genres, integration of real time payment.